DATA-260 HOMEWORK 6

Name	Ayushi Shringi
GitHub repository	https://github.com/ayushishringi/data260-0036
SID4	0036
PORT_BASE	8036 (8000 + 36 mod 900)
PREFIX	s0036
SEED	36
VERIFY_SEED	260036
DOMAIN_ID	4 (36 mod 8): open-source package vulnerability tracking
Hardware	MacBook Pro, Apple M5 Pro, 24 GB
Local model	Ollama qwen3:4b (num_ctx 16384, seed 36, temperature 0.2)
Tag	hw6
Commit	ce6d85494274131eb46762eee5b08639bd882a38

The GitHub repository link was verified, and both collaborators (Sbnikitha, supriyaselvanganesan) have access.

Part 1 - MongoDB Document API

1.1 Database and schema

Requirement	Field	Rule
Required short text (max 100)	title	required, 1-100 characters, not blank
Optional description	description	optional, max 5000 characters
Status-style enum with default	status	open | triaged | resolved | ignored, default open
Category-style fixed options	severity	critical | high | medium | low, default medium
Required date/time	discovered_at	required ISO 8601 datetime
Automatic timestamps	created_at, updated_at	set by the server on create/update

Validation is enforced twice: by Pydantic in the API and by a MongoDB $jsonSchema validator on the collection (strict, action=error).

Pydantic schema (backend/hw6_document_schemas.py)

```python

class DocumentInput(BaseModel):
    title: str = Field(min_length=1, max_length=100)
    description: str | None = Field(default=None, max_length=5000)
    severity: Severity = "medium"
    status: DocumentStatus = "open"
    discovered_at: datetime

    @field_validator("title")
    @classmethod
    def title_must_not_be_blank(cls, value: str) -> str:
        value = value.strip()
        if not value:
            raise ValueError("title cannot be blank")
        return value

    @field_validator("description")
    @classmethod
    def normalize_description(cls, value: str | None) -> str | None:
        return value.strip() if value is not None else None

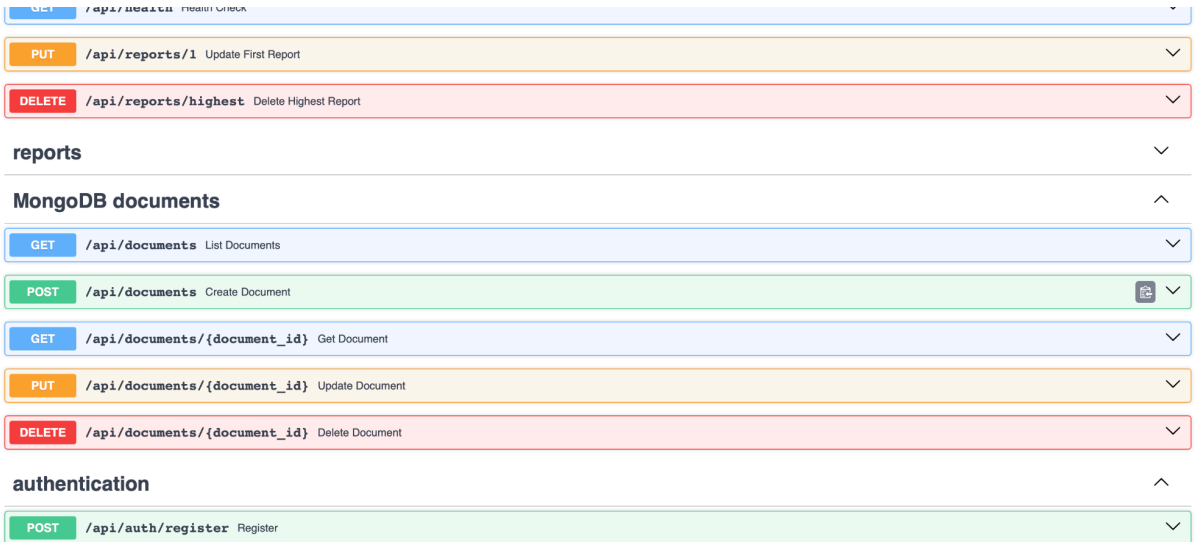
```

```
# backend/hw6_mongo_database.py, line 71
      "documents": {
      "$jsonSchema": {
         "bsonType": "object",
         "required": [
```

```
            "title", "severity", "status", "discovered_at",
            "created_at", "updated_at",
        ],
        "properties": {
            "title": {"bsonType": "string", "minLength": 1, "maxLength": 100},
            "description": {"bsonType": ["string", "null"]},
            "severity": {"enum": ["critical", "high", "medium", "low"]},
            "status": {"enum": ["open", "triaged", "resolved", "ignored"]},
            "discovered_at": {"bsonType": "date"},
            "created_at": {"bsonType": "date"},
            "updated_at": {"bsonType": "date"},
        },
    }},
```

1.2 API implementation (CRUD)

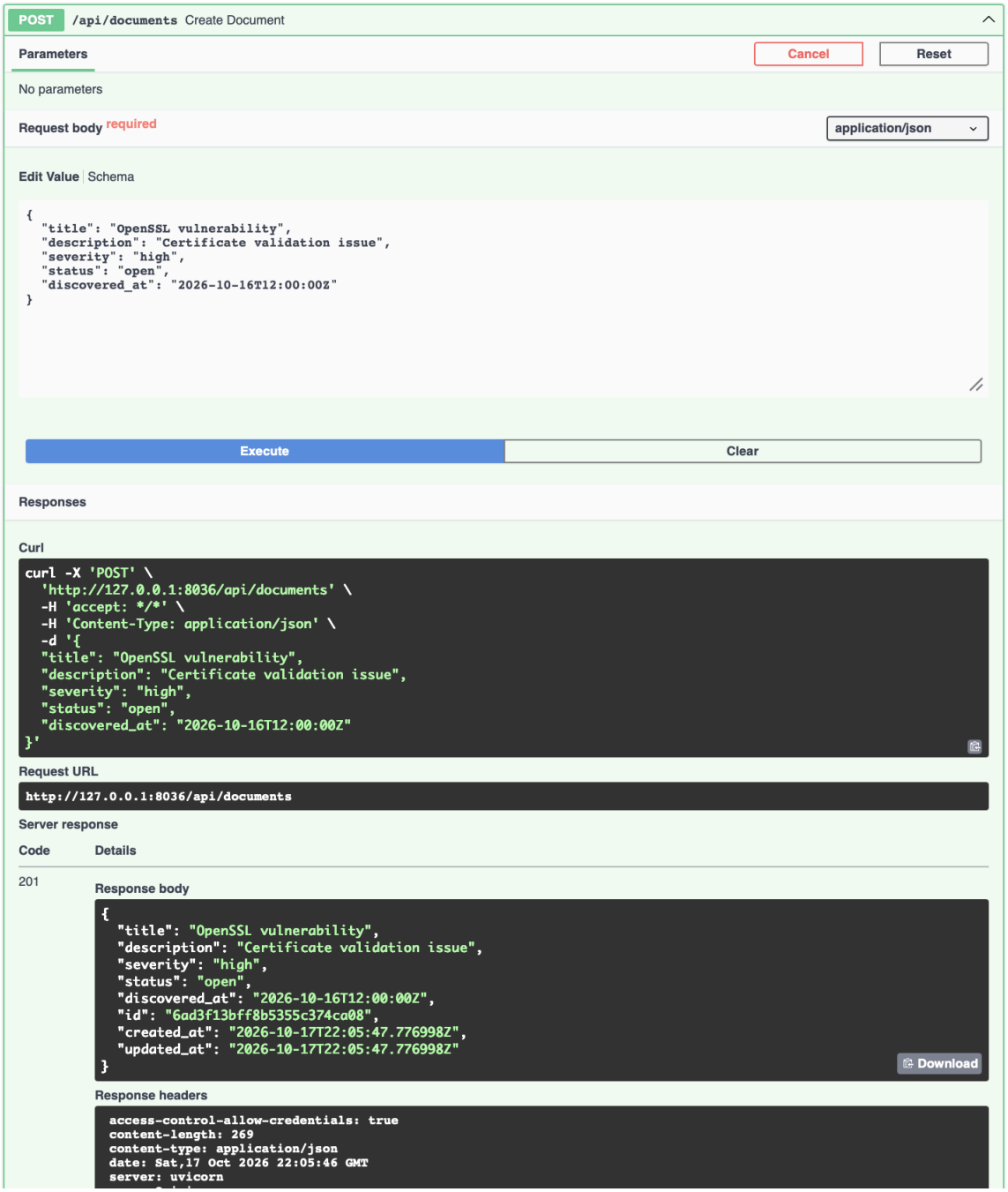
Swagger UI: the MongoDB documents routes.



POST handler: create.

```python
@router.post("", response_model=DocumentResponse, status_code=status.HTTP_201_CREATED)
def create_document(payload: DocumentInput) -> dict:
    collection = _collection_or_503()
    now = datetime.now(timezone.utc)
    document = {**payload.model_dump(), "created_at": now, "updated_at": now}
    try:
        result = collection.insert_one(document)
        document["_id"] = result.inserted_id
        return _to_response(document)
    except PyMongoError as exc:
        raise HTTPException(status_code=500, detail="Could not create document") from exc
```

POST /api/documents returns 201 with id and timestamps.



GET handlers: list all and get by id.

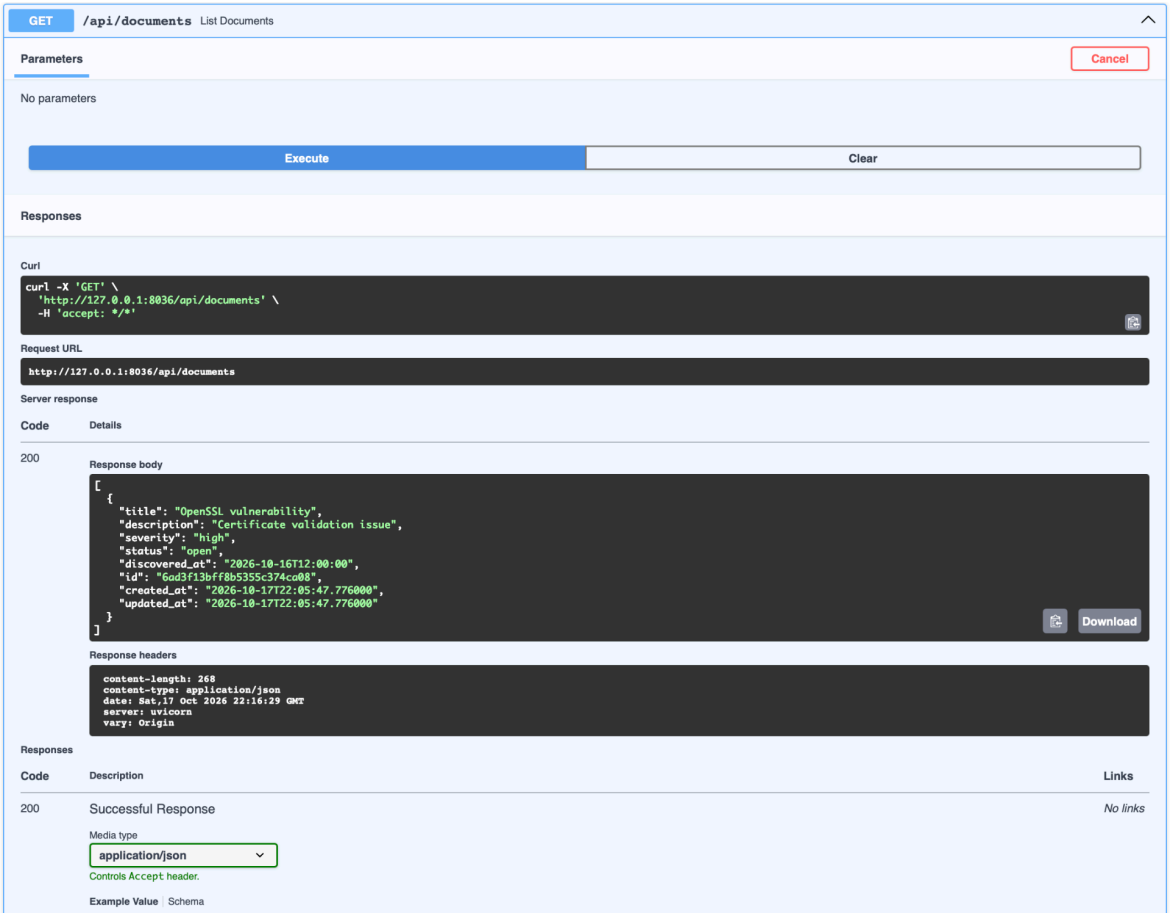
```python

@router.get("", response_model=list[DocumentResponse])
def list_documents() -> list[dict]:
    collection = _collection_or_503()
    try:
        return [_to_response(item) for item in collection.find().sort("created_at", -1)]
    except PyMongoError as exc:
        raise HTTPException(status_code=500, detail="Could not retrieve documents") from exc

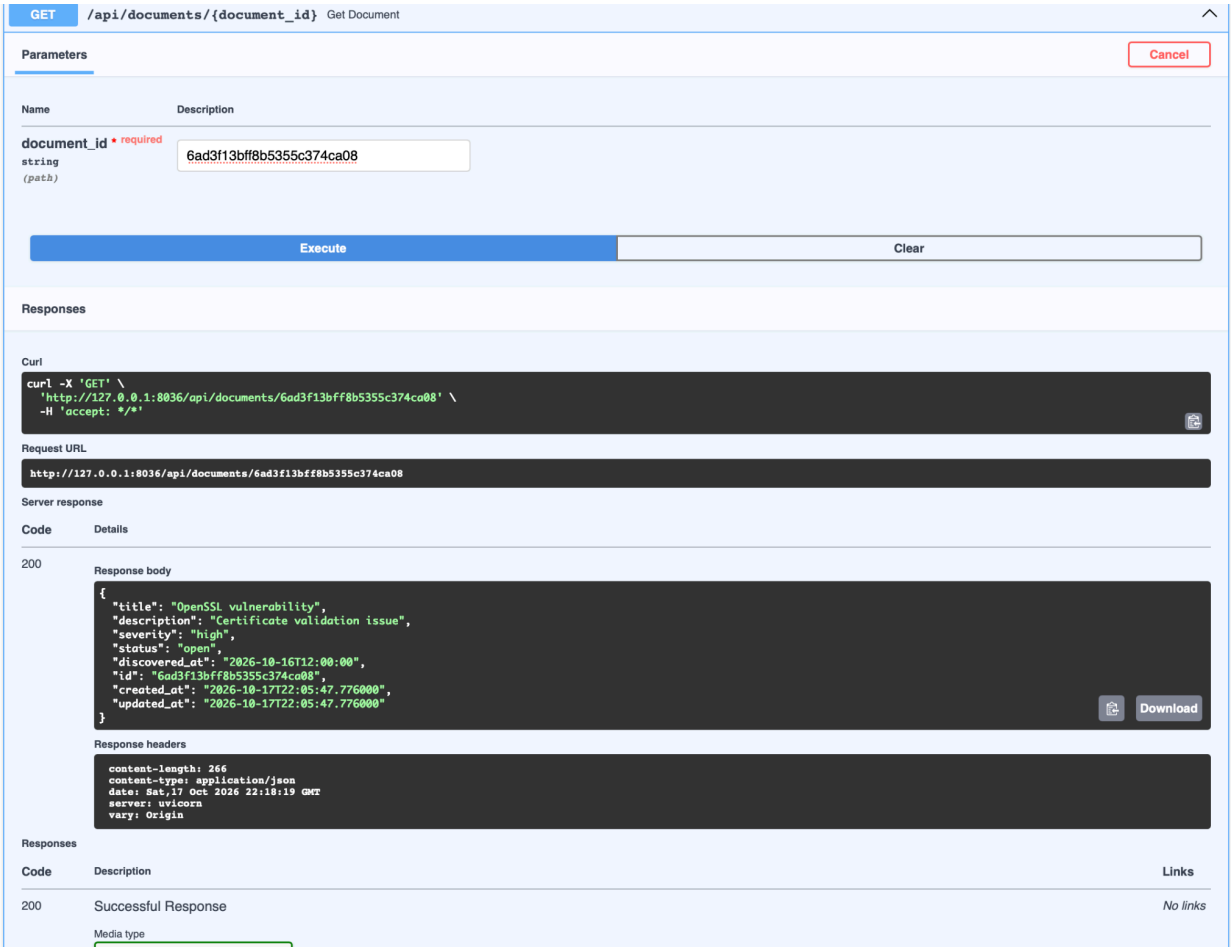

@router.get("/{document_id}", response_model=DocumentResponse)
def get_document(document_id: str) -> dict:
    collection = _collection_or_503()
    try:
        document = collection.find_one({"_id": _object_id(document_id)})
    except PyMongoError as exc:
        raise HTTPException(status_code=500, detail="Could not retrieve document") from exc
    if document is None:
        raise HTTPException(status_code=404, detail="Document not found")
    return _to_response(document)

```

GET /api/documents returns all records (200).



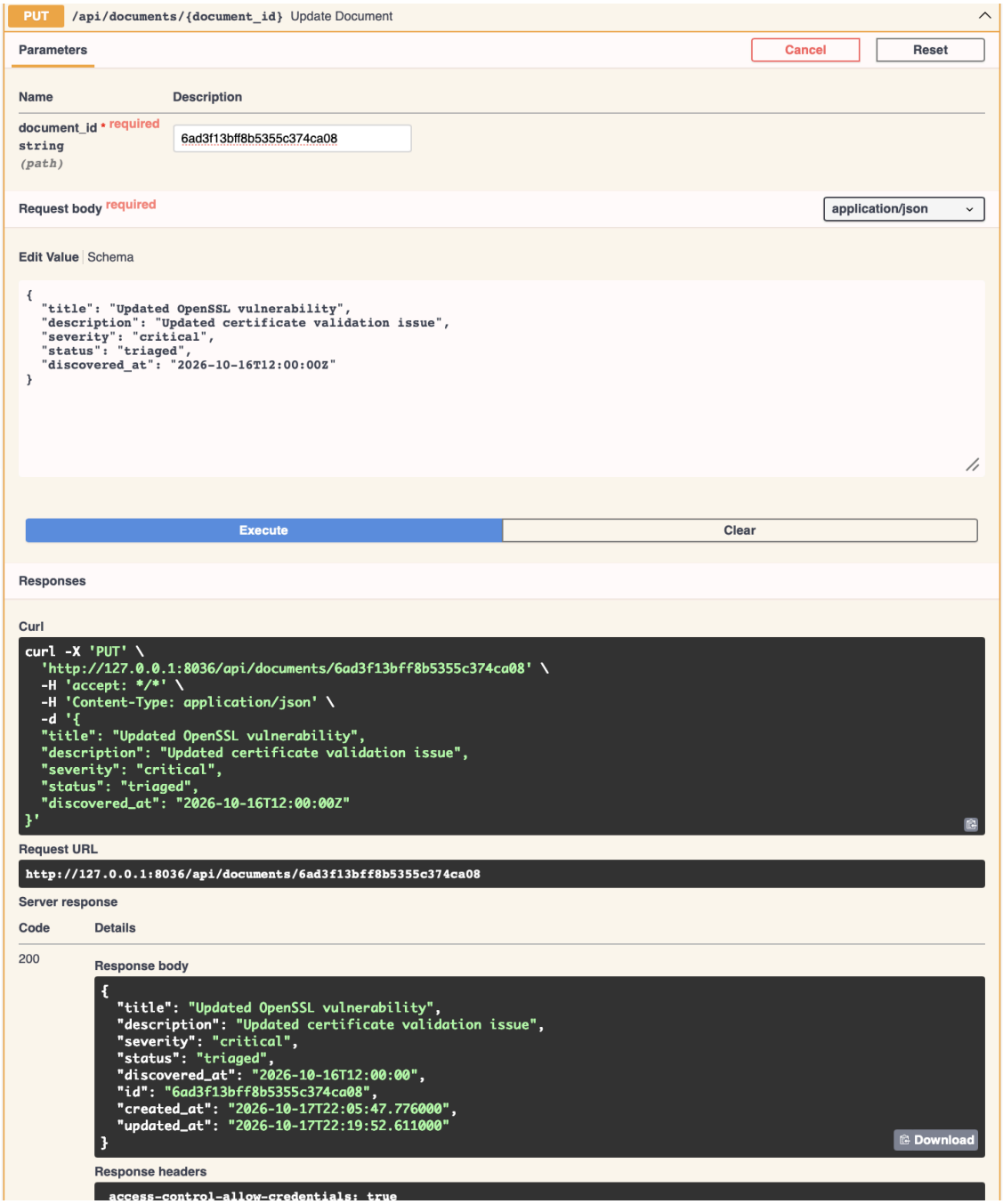
GET /api/documents/{id} returns one record (200).



PUT handler: update.

```python
@router.put("/{document_id}", response_model=DocumentResponse)
def update_document(document_id: str, payload: DocumentInput) -> dict:
    collection = _collection_or_503()
    object_id = _object_id(document_id)
    now = datetime.now(timezone.utc)
    try:
        result = collection.update_one(
            {"_id": object_id},
            {"$set": {**payload.model_dump(), "updated_at": now}},
        )
        if result.matched_count == 0:
            raise HTTPException(status_code=404, detail="Document not found")
        document = collection.find_one({"_id": object_id})
    except HTTPException:
        raise
    except PyMongoError as exc:
        raise HTTPException(status_code=500, detail="Could not update document") from exc
    return _to_response(document)
```

PUT /api/documents/{id}: severity and status updated, updated_at changed (200).



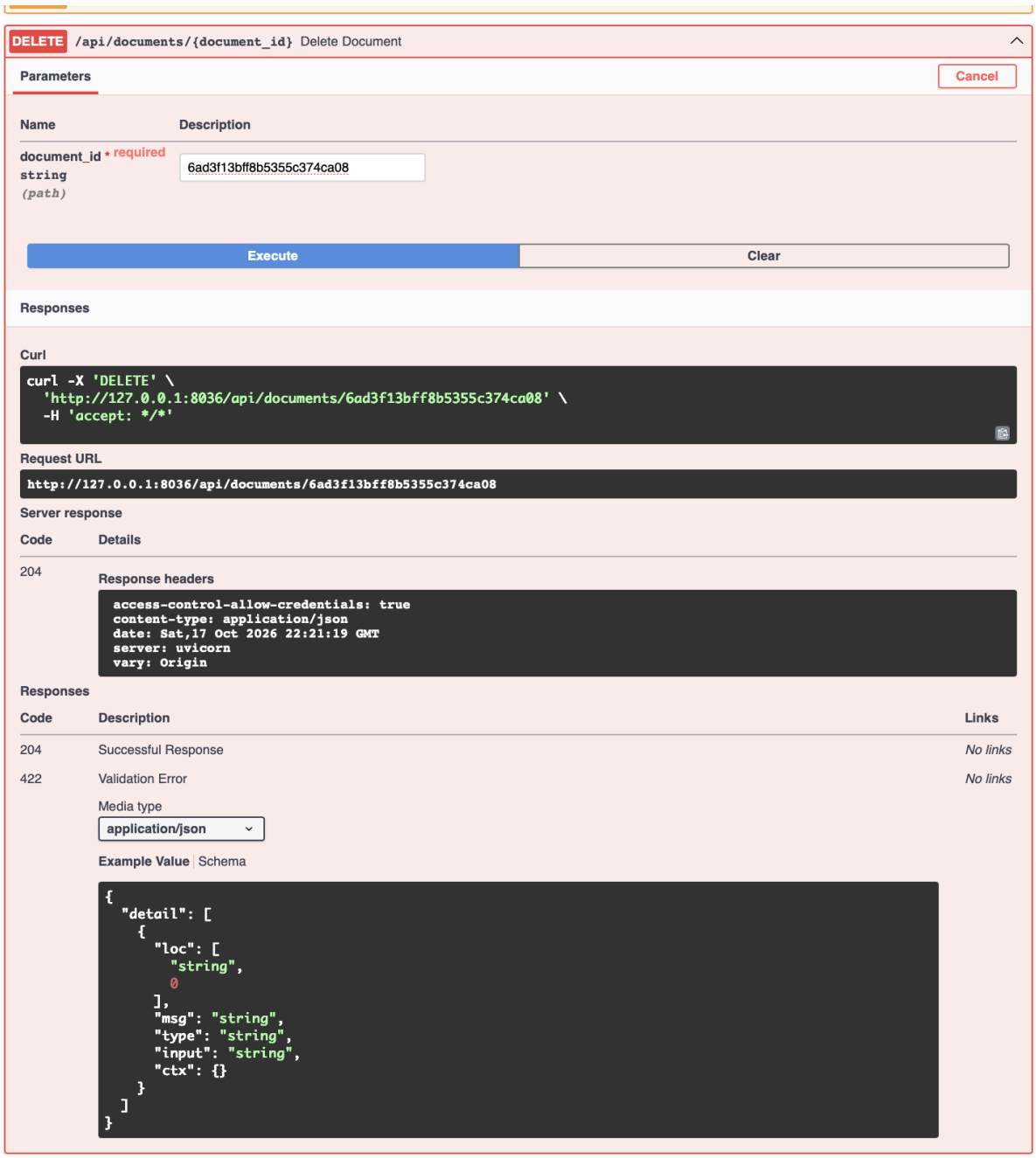
DELETE handler.

```python

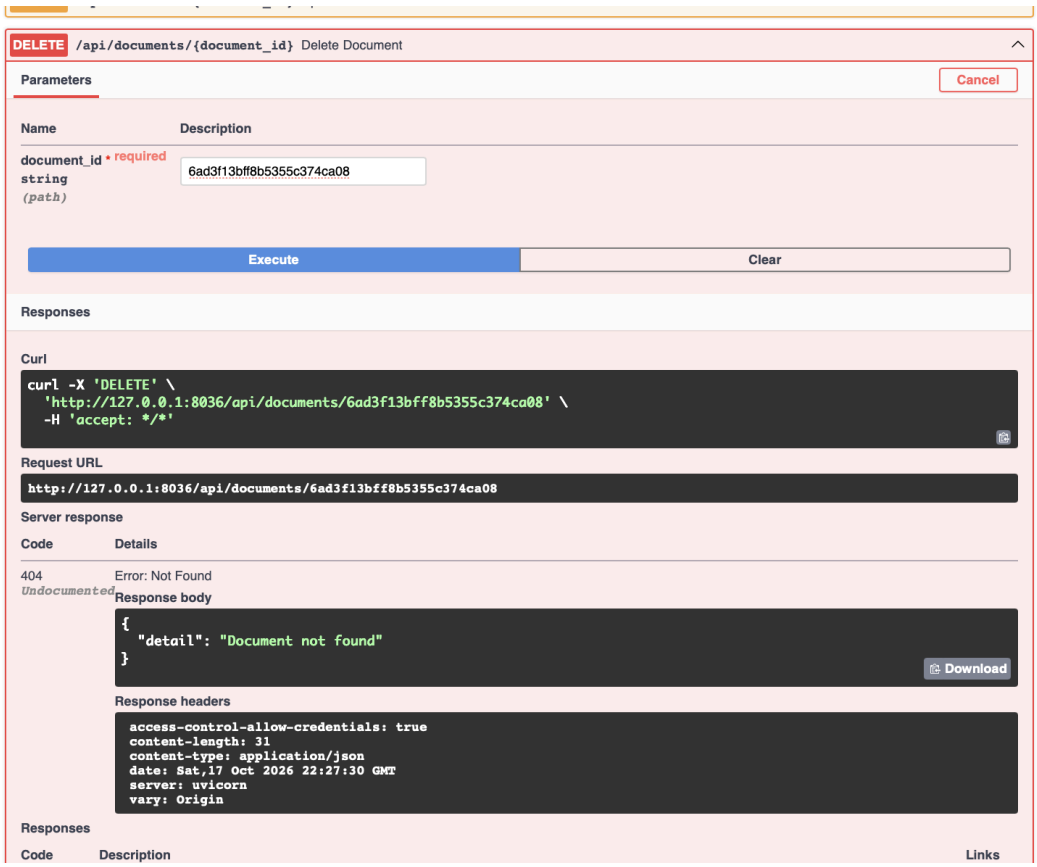
@router.delete("/{document_id}", status_code=status.HTTP_204_NO_CONTENT)
def delete_document(document_id: str) -> None:
    collection = _collection_or_503()
    try:
        result = collection.delete_one({"_id": _object_id(document_id)})
    except PyMongoError as exc:
        raise HTTPException(status_code=500, detail="Could not delete document") from exc
    if result.deleted_count == 0:
        raise HTTPException(status_code=404, detail="Document not found")

```

DELETE /api/documents/{id} returns 204.

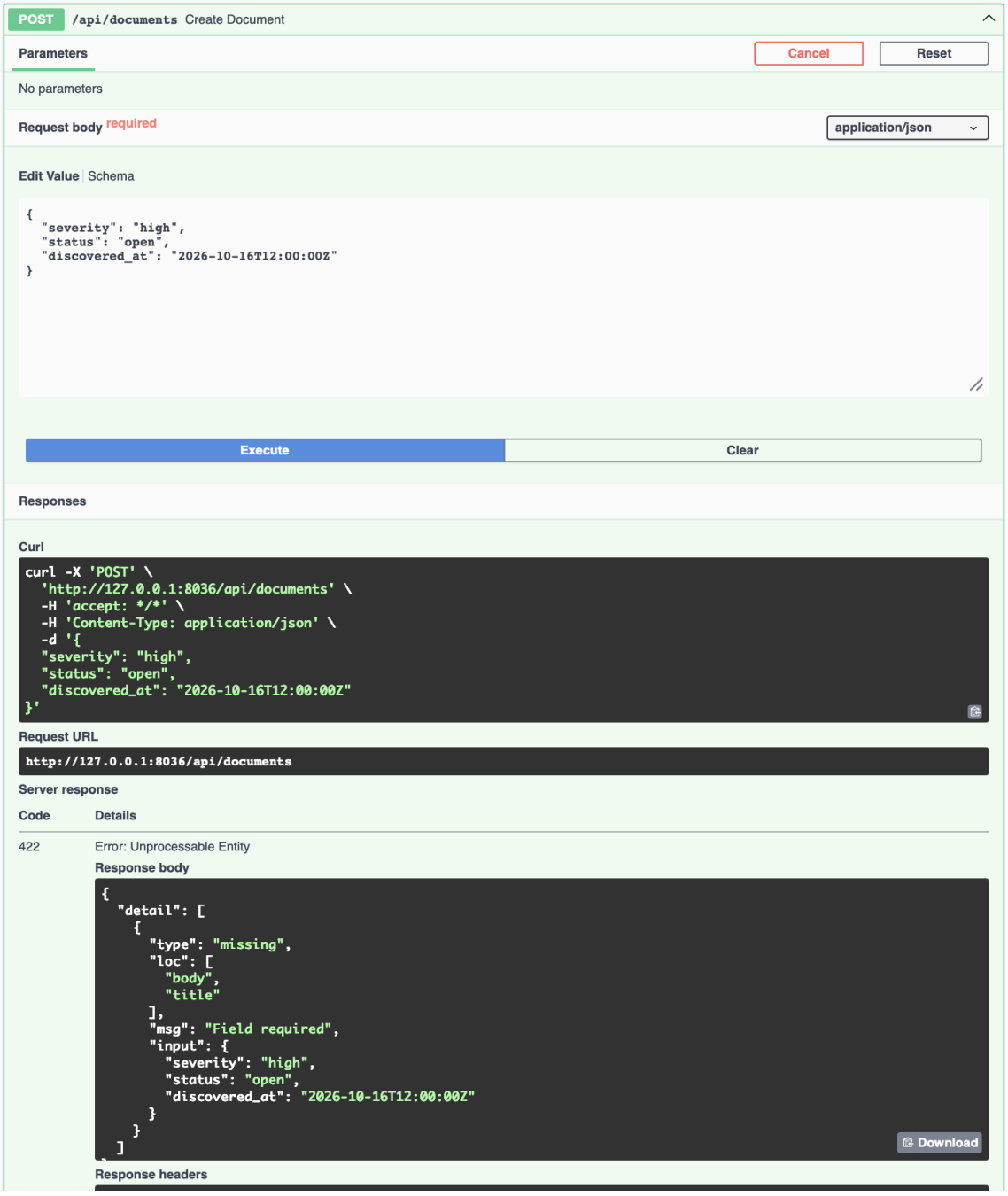


Deleting the same id again returns 404 Document not found.

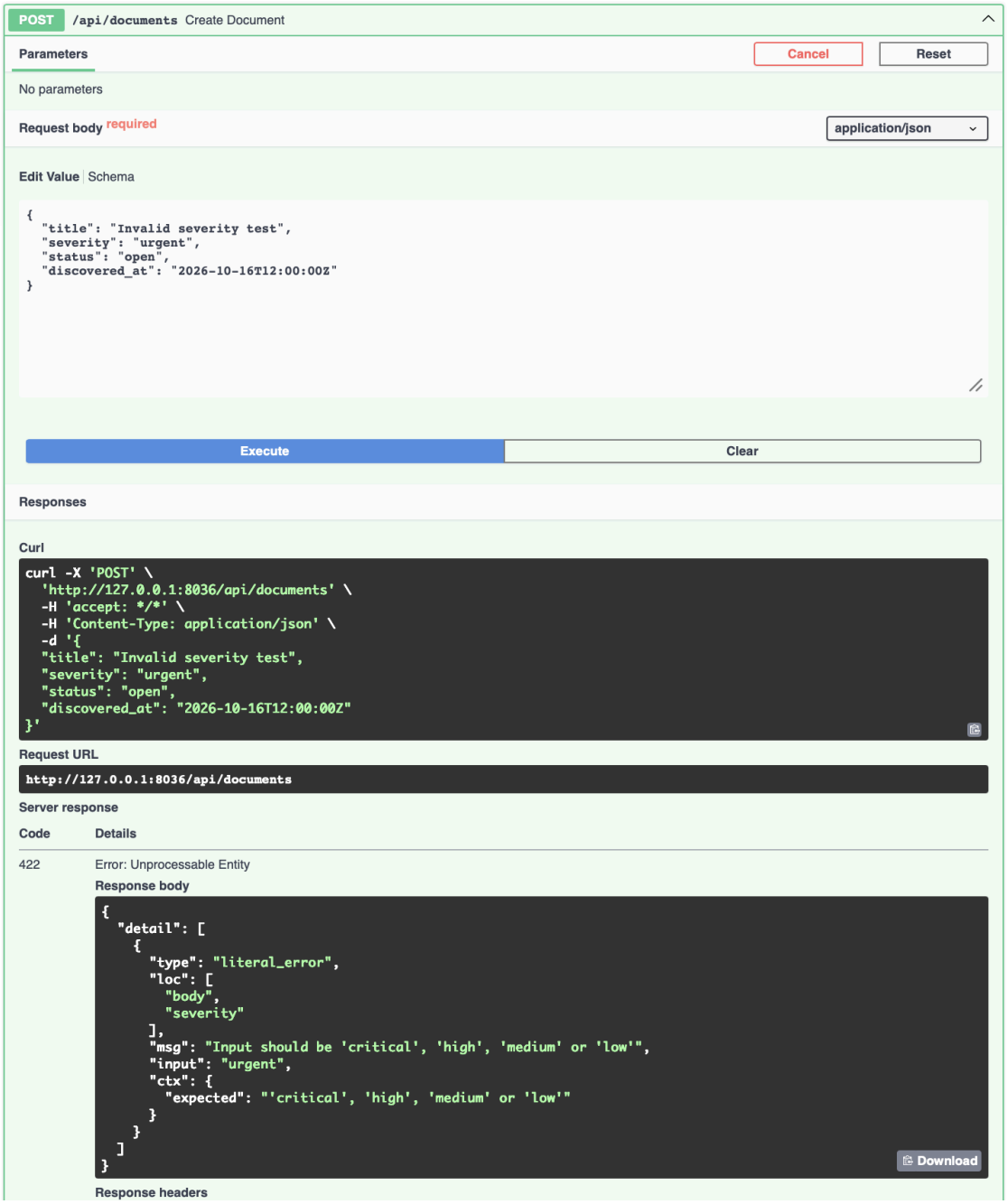


1.3 Data validation

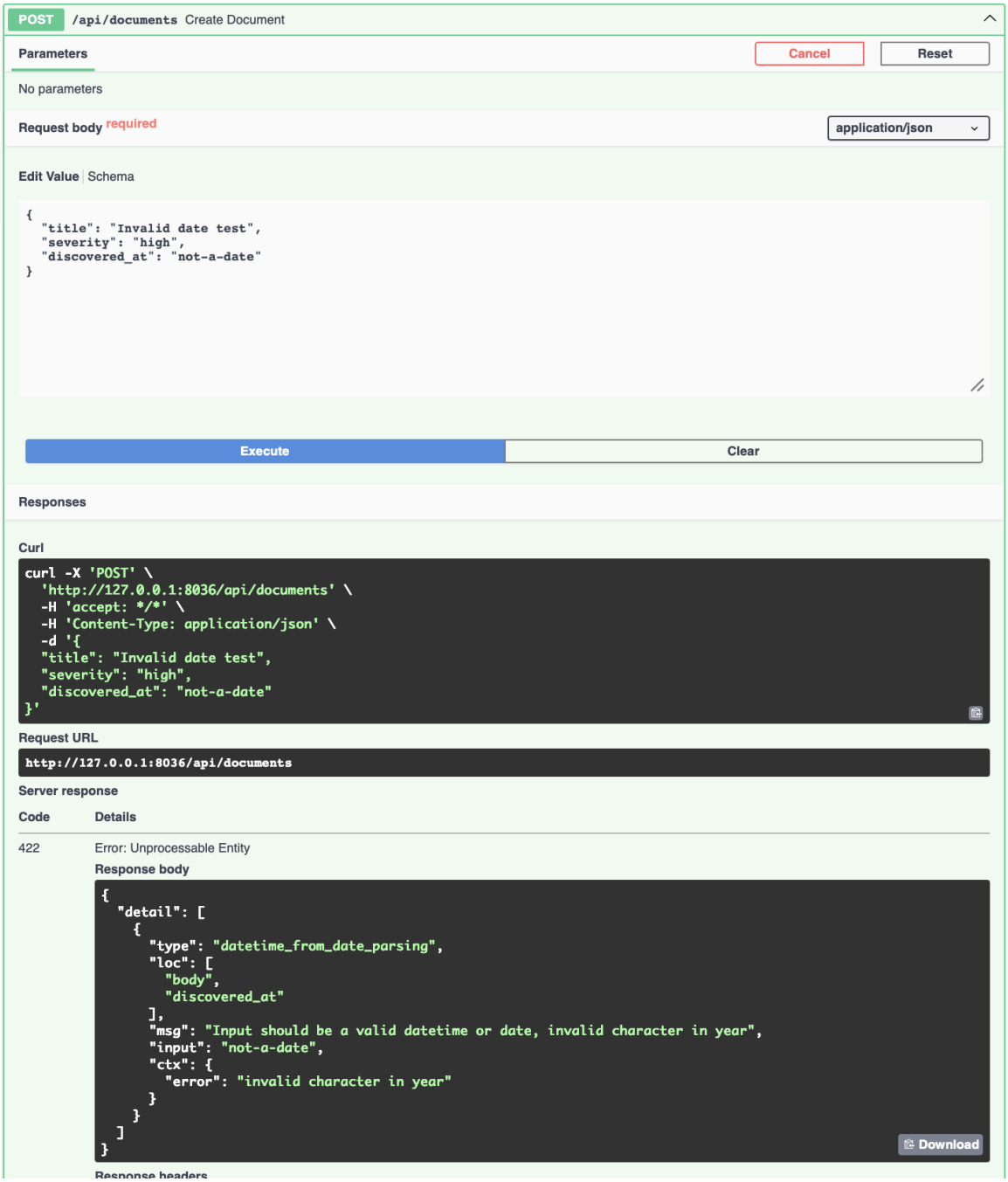
Missing required field (title) returns 422 Field required.



Invalid enum value severity='urgent' returns 422 listing allowed values.



Invalid date format returns 422.



SS 1.1: Incorrect data types (numeric title, non-date discovered_at) return 422.



1.4 Collection in MongoDB

mongosh: db.vulnerability_documents.find() showing sample data.

```
s0036_doc> db.vulnerability_documents.find().pretty()
[
  {
    _id: ObjectId('6ad3f7bdff8b5355c374ca09'),
    title: 'MongoDB sample vulnerability',
    description: 'Sample document for HW6 evidence',
    severity: 'medium',
    status: 'open',
    discovered_at: ISODate('2026-10-16T12:00:00.000Z'),
    created_at: ISODate('2026-10-17T22:33:33.458Z'),
    updated_at: ISODate('2026-10-17T22:33:33.458Z')
  }
]
s0036_doc>
```

Part 2 - Short-, Long-, and Episodic Memory

2.1 Collections

The same database s0036_doc and connection are reused,the Part 1 collection is unchanged. Three new collections each have a $jsonSchema validator:

- **messages:** user_id, session_id, role (user | assistant), content, created_at.
- **summaries:** user_id, session_id (null for lifetime), scope ('session' | 'user'), text, created_at.
- **episodes:** user_id, session_id, fact (max 200 chars), importance (0..1), embedding (384 floats, all-MiniLM-L6-v2), created_at.

Settings: SHORT_TERM_N=12, SUMMARIZE_EVERY_USER_MSGS=5, lifetime summary every 2 session summaries, EPISODIC_TOP_K=3.

2.2 Endpoint code

```
# backend/hw6_memory_api.py, line 14
class ChatRequest(BaseModel):
    user_id: str = Field(min_length=1)
    session_id: str | None = None
    message: str = Field(min_length=1)


@router.post("/chat")
def chat(payload: ChatRequest) -> dict:
    try:
        return make_chat_turn(
            payload.user_id.strip(),
            payload.session_id.strip() if payload.session_id else None,
            payload.message.strip(),
            ModelClient(),
        )
    except (PyMongoError, RuntimeError) as exc:
        raise HTTPException(status_code=503, detail=str(exc)) from exc


@router.get("/memory/{user_id}")
def memory(user_id: str) -> dict:
    try:
        return get_memory(user_id.strip())
    except PyMongoError as exc:
        raise HTTPException(status_code=503, detail="MongoDB is unavailable") from exc


@router.get("/aggregate/{user_id}")
def aggregate(user_id: str) -> dict:
```

```
    try:
        return get_aggregate(user_id.strip())
    except PyMongoError as exc:
        raise HTTPException(status_code=503, detail="MongoDB is unavailable") from exc
```

```
# backend/hw6_memory_service.py, line 346
def make_chat_turn(
    user_id: str,
    session_id: str | None,
    user_message: str,
    model_client: ModelClient,
) -> dict[str, Any]:
    started = time.perf_counter()
    command_result = handle_file_memory_command(
        user_message,
        user_id,
        session_id,
        model_client,
    )
    if command_result is not None:
        save_message(user_id, session_id, "user", user_message)
        reply = command_result.get("message", "")
        save_message(user_id, session_id, "assistant", reply)
        latency_ms = round((time.perf_counter() - started) * 1000, 3)
        return {
            "reply": reply,
            "file_memory": command_result,
            "used": {"short_term_messages": [], "long_term_summary": None, "episodic_facts": []},
            "usage": {
                "prompt_tokens": 0,
                "completion_tokens": 0,
                "total_tokens": 0,
                "latency_ms": latency_ms,
                "component_token_estimates": {},
            },
        }
    auto_compaction = compact_memory() if consolidation_required() else None
    save_message(user_id, session_id, "user", user_message)
    short_term = load_short_term_messages(user_id, session_id)
    summaries = load_latest_summaries(user_id, session_id)
    episodes = retrieve_episodes(user_id, user_message)
    prior_messages = short_term[:-1] if short_term and short_term[-1]["role"] == "user" else
short_term
    prompt = assemble_prompt(user_message, prior_messages, summaries, episodes)
    result = model_client.complete(prompt)
```

```
    save_message(user_id, session_id, "assistant", result.text)
    extract_episodes(user_id, session_id, user_message)
    maybe_summarize(user_id, session_id, model_client)
    ...  # token accounting and response (see Part 3)
```

Besides the reply, the response contains usage (short-term message count and list, the long-term summary, and the episodic facts with importance and similarity) and usage (tokens, latency, and the active memory config).

2.3 Memory logic functions

Short-term window. Short-term memory: the last SHORT_TERM_N messages of the session.

```
# backend/hw6_memory_service.py
def load_short_term_messages(user_id: str, session_id: str | None) -> list[dict[str, Any]]:
    query = {"user_id": user_id, "session_id": session_id}
    rows = list(
        get_messages_collection()
        .find(query)
        .sort("created_at", -1)
        .limit(SHORT_TERM_N)
    )
    rows = list(reversed(rows))
    if MEMORY_POLICY == "compressed_window":
        rows = compress_window(rows)
    return rows
```

Episode extraction. Up to 3 declarative sentences per user message are stored as facts (questions are skipped). Each gets a heuristic importance score and an embedding.

```
# backend/hw6_memory_service.py
def score_importance(fact: str) -> float:
    """Heuristic 0..1 importance: personal, specific, durable statements score higher."""

    text = fact.casefold()
    score = 0.3
    if IMPORTANCE_PERSONAL_RE.search(text):
        score += 0.3
    if re.search(r"\d", text):
        score += 0.2
    if IMPORTANCE_DURABLE_RE.search(text):
        score += 0.2
    return round(min(score, 1.0), 2)

# backend/hw6_memory_service.py
def extract_episodes(user_id: str, session_id: str | None, message: str) -> list[dict[str, Any]]:
    """Store up to 3 declarative sentences from a user message as scored episodic facts.
```

```
    Questions are not facts, so they are skipped.
    """

    sentences = [part.strip() for part in re.split(r"(?<=[.!?])\s+|\n+", message) if part.strip()]
    candidates = [
        sentence.rstrip(".!").strip()
        for sentence in sentences
        if not sentence.endswith("?") and len(sentence.split()) >= 3
    ]
    scored = sorted(
        ((score_importance(fact), fact) for fact in candidates),
        key=lambda item: item[0],
        reverse=True,
    )[:3]
    collection = get_episodes_collection()
    saved = []
    for importance, fact in scored:
        if len(fact) > 200:
            fact = fact[:197] + "..."
        document = {
            "user_id": user_id,
            "session_id": session_id,
            "fact": fact,
            "importance": float(importance),
            "embedding": embed_text(fact),
            "created_at": now_utc(),
        }
        result = collection.insert_one(document)
        document["_id"] = result.inserted_id
        saved.append(document)
    return saved
```

Embedding and retrieval. The current message is embedded, and the top-k episodes are returned by cosine similarity.

```
# backend/hw6_memory_service.py
def embed_text(text: str) -> list[float]:
    """Create a local embedding, falling back to a deterministic vector."""

    try:
        from sentence_transformers import SentenceTransformer

        if not hasattr(embed_text, "_model"):
            embed_text._model = SentenceTransformer("all-MiniLM-L6-v2")
```

```
        return embed_text._model.encode(text, normalize_embeddings=True).astype(float).tolist()
    except Exception:
        return _hash_embedding(text)

# backend/hw6_memory_service.py
def cosine_similarity(left: list[float], right: list[float]) -> float:
    left_array = np.asarray(left, dtype=float)
    right_array = np.asarray(right, dtype=float)
    denominator = np.linalg.norm(left_array) * np.linalg.norm(right_array)
    return float(np.dot(left_array, right_array) / denominator) if denominator else 0.0

# backend/hw6_memory_service.py
def retrieve_episodes(user_id: str, query: str) -> list[dict[str, Any]]:
    query_embedding = embed_text(query)
    scored: list[tuple[float, dict[str, Any]]] = []
    for episode in get_episodes_collection().find({"user_id": user_id}):
        score = cosine_similarity(query_embedding, episode.get("embedding", []))
        scored.append((score, episode))
    scored.sort(key=lambda item: item[0], reverse=True)
    return [{**episode, "similarity": score} for score, episode in scored[:EPISODIC_TOP_K]]
```

Summarization trigger. Every 5th user message summarizes the unsummarized part of the session (scope='session'). Every 2nd session summary is condensed into a lifetime summary (scope='user', session_id=null).

```
# backend/hw6_memory_service.py
def maybe_summarize(
    user_id: str,
    session_id: str | None,
    model_client: ModelClient,
) -> None:
    user_count = get_messages_collection().count_documents({
        "user_id": user_id,
        "session_id": session_id,
        "role": "user",
    })
    if user_count == 0 or user_count % SUMMARIZE_EVERY_USER_MSGS != 0:
        return
    messages = load_messages_since_last_summary(user_id, session_id)
    text = _summary_text(messages, model_client)
    collection = get_summaries_collection()
    collection.insert_one({
        "user_id": user_id,
        "session_id": session_id,
        "scope": "session",
```

```
        "text": text,
        "created_at": now_utc(),
    })
    session_summaries = list(collection.find({
        "user_id": user_id,
        "scope": "session",
    }).sort("created_at", -1))
    if len(session_summaries) % LIFETIME_SUMMARIZE_EVERY_SESSION_SUMMARIES !=
0:
        return
    lifetime_input = "\n\n".join(item["text"] for item in session_summaries)
    lifetime_result = model_client.complete([
        {
            "role": "system",
            "content": (
                "Condense these session summaries into at most 8 lifetime memory bullets. "
                "Keep every concrete fact the user stated. Do not add advice."
            ),
        },
        {"role": "user", "content": lifetime_input},
    ])
    collection.insert_one({
        "user_id": user_id,
        "session_id": None,
        "scope": "user",
        "text": lifetime_result.text,
        "created_at": now_utc(),
    })
```

Prompt assembly. System primer + file memory + lifetime/session summaries + episodic facts, then the short-term window and the current message.

```
# backend/hw6_memory_service.py
def assemble_prompt(
    user_message: str,
    short_term: list[dict[str, Any]],
    summaries: dict[str, Any],
    episodes: list[dict[str, Any]],
) -> list[dict[str, str]]:
    long_term = "\n".join(
        item["text"] for item in (summaries.get("user"), summaries.get("session")) if item
    ) or "(none)"
    episode_text = "\n".join(f"- {item['fact']}" for item in episodes) or "(none)"
    context = (
        f"Lifetime/session summaries:\n{long_term}\n\n"
```

```
        f"Relevant episodic facts:\n{episode_text}"
    )
    messages = [{
        "role": "system",
        "content": SYSTEM_PRIMER + "\n\nFile-based memory:\n" + file_memory_context() +
"\n\n" + context,
    }]
    messages.extend({"role": item["role"], "content": item["content"]} for item in short_term)
    messages.append({"role": "user", "content": user_message})
    return messages
```

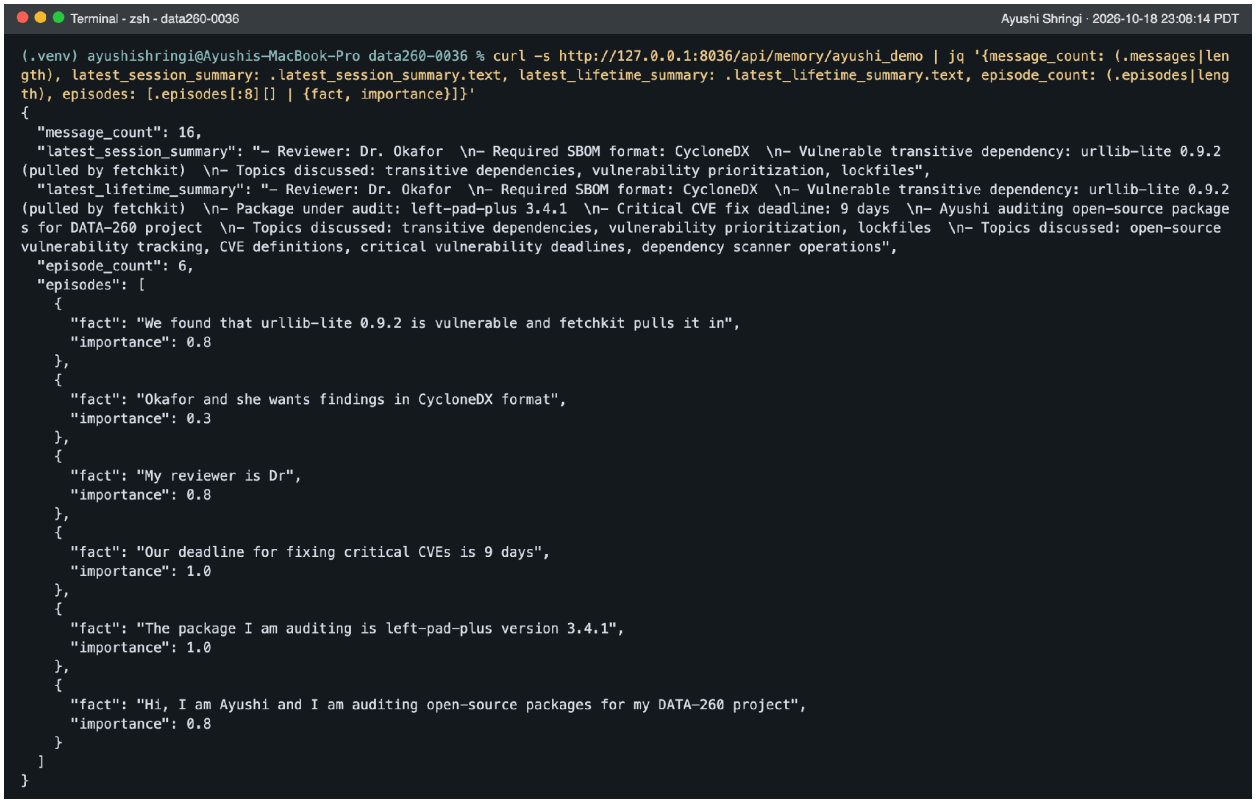
2.4 Conversation (12 messages in one session)

Twelve POST /api/chat calls in one session, showing replies, short-term window size, summary use and retrieved episodic facts per turn.



2.5 GET /api/memory/{user_id}

Last 16 messages, latest session and lifetime summaries, and last 20 episodes.



2.6 GET /api/aggregate/{user_id}

Daily message counts grouped by date and the most recent summaries.

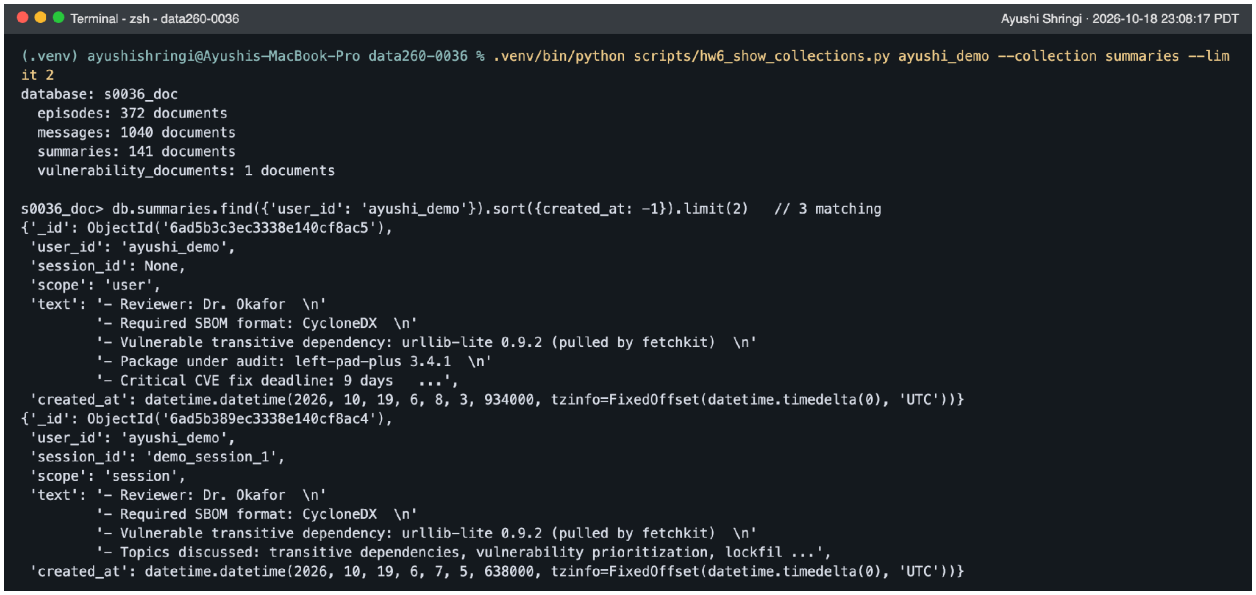


2.7 MongoDB collections

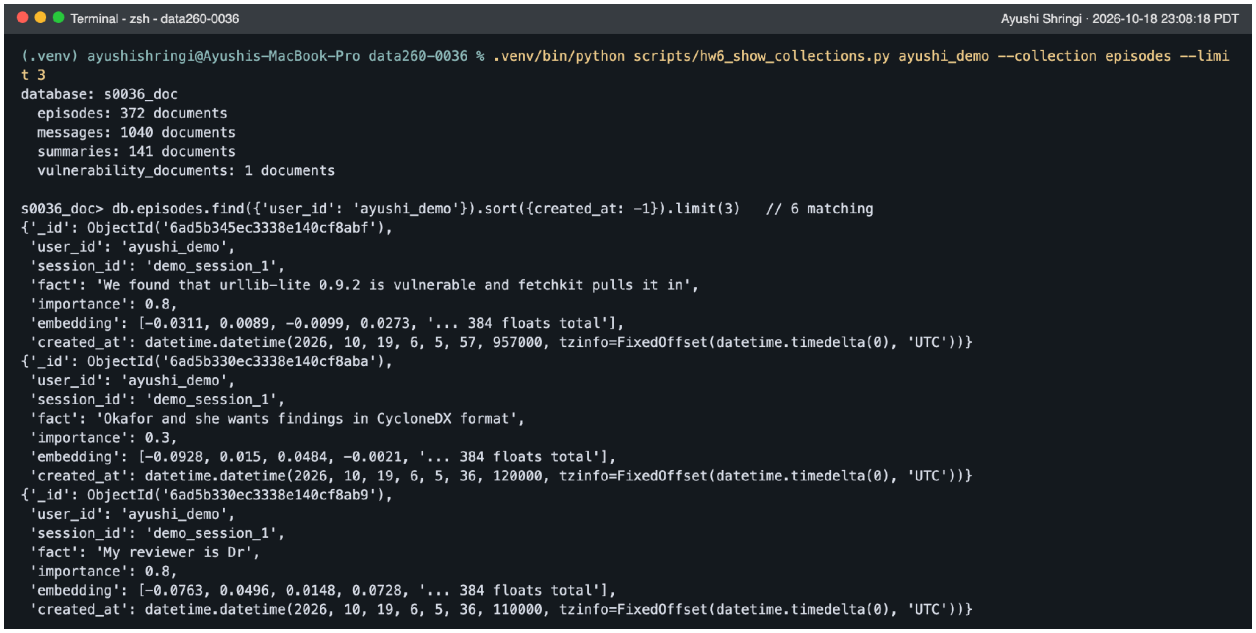
messages collection for ayushi_demo (PyMongo query; mongosh is not installed).



summaries collection for ayushi_demo (session and lifetime scope).



episodes collection for ayushi_demo (fact, importance, embedding).



Part 3 - What Memory Costs

3.1 Logging memory usage

Each /api/chat request records prompt and completion tokens (Ollama prompt_eval_count / eval_count), estimated tokens for the system primer, file memory, long-term summary, short-term window and episodic facts (characters / 4), latency, and the active memory config. Records are appended to reports/hw06/raw/chat_costs.jsonl, and the experiment runner also writes raw/per_turn_tokens.csv and raw/memory_cost_comparison.json.

```
# backend/hw6_memory_service.py
   cost_record = {
       "timestamp": now_utc().isoformat(),
       "user_id": user_id,
       "session_id": session_id,
       "prompt_tokens": result.input_tokens,
       "completion_tokens": result.output_tokens,
       "total_tokens": result.total_tokens,
       "component_token_estimates": {
           "system_primer": estimate_tokens(SYSTEM_PRIMER),
           "file_memory": estimate_tokens(file_memory_context()),
           "long_term_summary": estimate_tokens(summary_text),
           "short_term_window": estimate_tokens(short_term_text),
           "episodic_facts": estimate_tokens(episodic_text),
       },
       "latency_ms": latency_ms,
       "config": memory_config(),
   }
   log_chat_cost(cost_record)
```

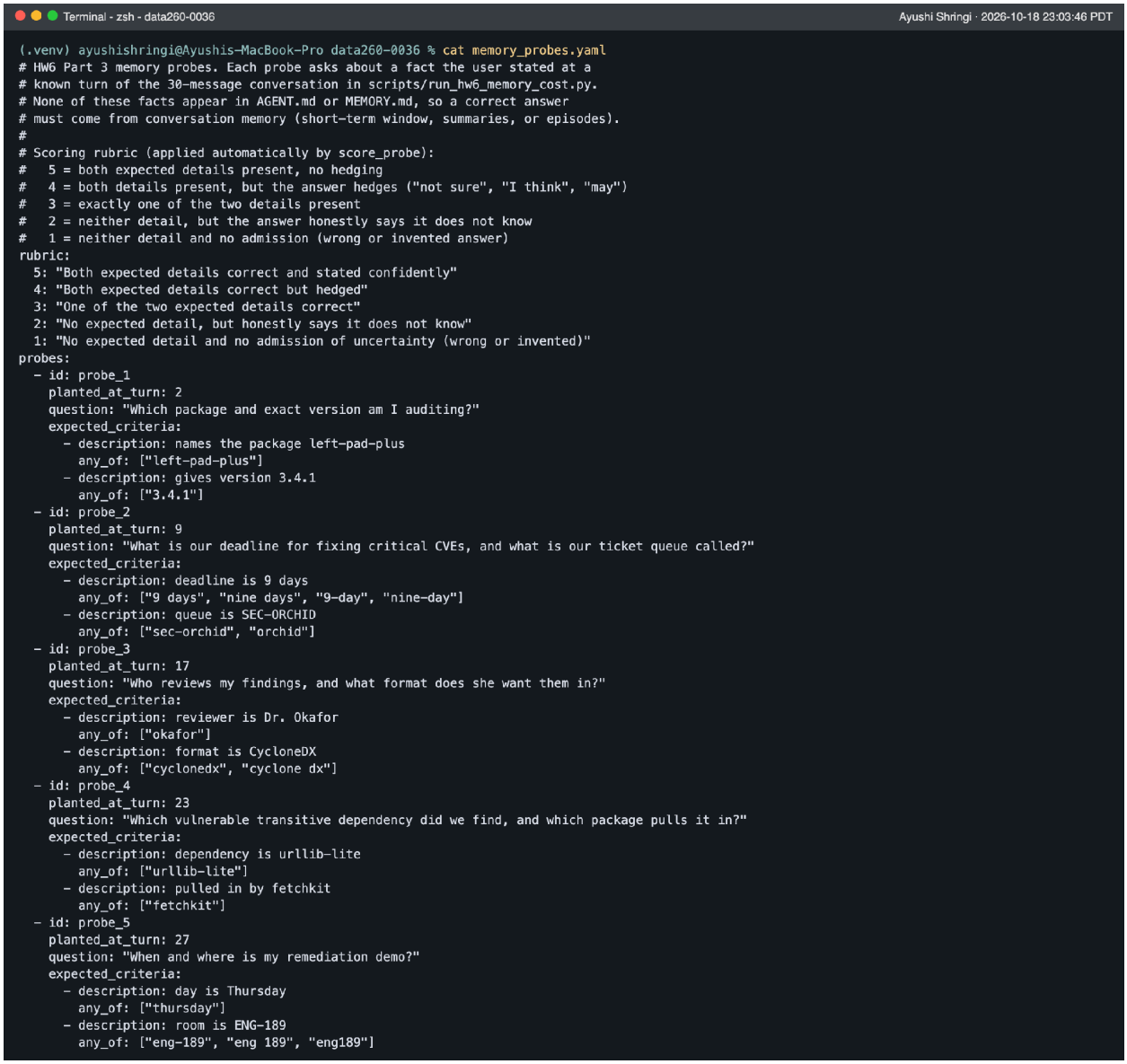
One logged /api/chat request from raw/chat_costs.jsonl.



3.2 Memory probes

memory_probes.yaml was written before the runs. Each of its five probes asks about a fact the user stated at turn 2, 9, 17, 23 or 27 of the 30-message conversation. None of these facts appear in AGENT.md or MEMORY.md, so only conversation memory can answer them. Each probe has two expected details, scored on this rubric: 5 = both details; 4 = both but hedged; 3 = one detail; 2 = neither but admits not knowing; 1 = neither and no admission.

memory_probes.yaml with five questions, expected criteria and the 1-5 rubric.



3.3 Comparing short-term memory sizes

scripts/run_hw6_memory_cost.py starts a fresh API server for each configuration with SHORT_TERM_N and MEMORY_POLICY set as environment variables. It aborts if /api/chat

reports a different config. It then runs the same 30-message conversation in a fresh session and asks the five probes.

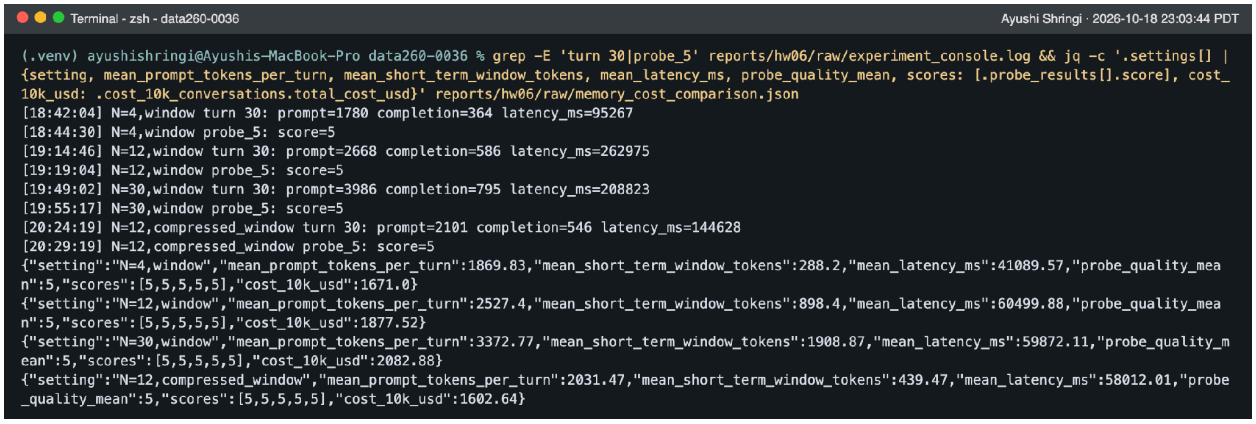
```
# scripts/run_hw6_memory_cost.py
def score_probe(reply: str, probe: dict) -> int:
    """Apply the 1-5 rubric in memory_probes.yaml to one reply."""

    text = reply.casefold()
    criteria = probe["expected_criteria"]
    hits = sum(1 for criterion in criteria if any(word in text for word in criterion["any_of"]))
    if hits == len(criteria):
        return 4 if any(word in text for word in HEDGE_WORDS) else 5
    if hits:
        return 3
    return 2 if any(word in text for word in UNKNOWN_WORDS) else 1
```

Setting	Mean prompt tokens/turn	Mean latency (ms)	Probe quality (1-5)
SHORT_TERM_N=4	1,869.8	41,090	5.0
SHORT_TERM_N=12	2,527.4	60,500	5.0
SHORT_TERM_N=30	3,372.8	59,872	5.0
Compression policy (N=12, compressed_window)	2,031.5	58,012	5.0

Setting	Short-term window tokens/turn	Messages in window	Probe scores (turns 2, 9, 17, 23, 27)
SHORT_TERM_N=4	288	3.9	5, 5, 5, 5, 5
SHORT_TERM_N=12	898	10.8	5, 5, 5, 5, 5
SHORT_TERM_N=30	1,909	22.5	5, 5, 5, 5, 5
Compression policy (N=12, compressed_window)	439	10.8	5, 5, 5, 5, 5

Experiment output: summary of the four settings.



Findings. Window size drives input cost: short-term-window tokens grew from 288 (N=4) to 898 (N=12) to 1,909 (N=30) per turn, and mean prompt tokens from 1,870 to 2,527 to 3,373.

Probe quality was 5.0 in every setting. In every run, episodic retrieval returned the planted fact as the top match for each probe, even at N=4 where all five facts were far outside the 4-message window. With episodic memory enabled, a larger window added cost without improving recall on these probes.

Latency is not a reliable ranking signal here. Summarization runs inside every 5th request (50-260 s on those turns), and the settings ran one after another on one laptop, so later runs were slower.

3.4 Memory-saving policy: compressed_window

The policy keeps the last N messages but cuts every assistant reply except the most recent to 240 characters. User messages stay verbatim, and the session summary and episodic facts carry the older content.

```
# backend/hw6_memory_service.py
def compress_window(rows: list[dict[str, Any]]) -> list[dict[str, Any]]:
    """Shorten older assistant replies; user messages are kept verbatim."""

    assistant_indexes = [index for index, row in enumerate(rows) if row["role"] == "assistant"]
    keep_full = assistant_indexes[-1] if assistant_indexes else None
    compressed = []
    for index, row in enumerate(rows):
        content = row["content"]
        if row["role"] == "assistant" and index != keep_full and len(content) >
COMPRESSED_REPLY_CHARS:
            content = content[:COMPRESSED_REPLY_CHARS].rsplit(" ", 1)[0] + " [...]"
        compressed.append({**row, "content": content})
    return compressed
```

Metric	N=12, window	N=12, compressed_window	Change
Mean prompt tokens/turn	2,527.4	2,031.5	-19.6%
Short-term window tokens/turn	898.4	439.5	-51.1%
Input tokens per conversation	75,822	60,944	-19.6%
Mean latency (ms)	60,500	58,012	-4.1%
Probe quality (1-5)	5.0	5.0	no change

Compared with the same N=12 window, compression cut short-term-window tokens by 51.1% and prompt tokens by 19.6%, with no change in probe quality.

3.5 Estimated cost of 10,000 30-turn conversations

Cost = 10,000 x (input tokens x $1 / 1M + output tokens x $5 / 1M), using the measured tokens of each 30-turn run (probes excluded).

Setting	Input tok/conv	Output tok/conv	Input cost	Output cost	Total cost	$ per quality point
N=4	56,095	22,201	$560.95	$1,110.05	**$1,671.00**	$334.20
N=12	75,822	22,386	$758.22	$1,119.30	**$1,877.52**	$375.50
N=30	101,183	21,421	$1,011.83	$1,071.05	**$2,082.88**	$416.58
Compressed (N=12, compressed_window)	60,944	19,864	$609.44	$993.20	**$1,602.64**	$320.53

How the best and worst settings were selected. Settings were ranked by total cost divided by mean probe quality (dollars per quality point). A setting therefore only counts as best if it is cheap without losing recall.

- **Best:** N=12 compressed_window: $1,602.64 for 10,000 conversations at quality 5.0.
- **Worst:** SHORT_TERM_N=30: $2,082.88 at the same quality, which is $480.24 (30%) more.

On input tokens alone, N=4 is slightly cheaper than the compressed policy (56,095 vs 60,944). The compressed run's total is lower partly because its replies happened to be shorter, and output is priced 5x higher. The two are therefore effectively tied on cost. If recall of recent detail matters, the compressed N=12 window is preferable, because it keeps 12 messages for about the cost of 4.

3.6 Note on the earlier run

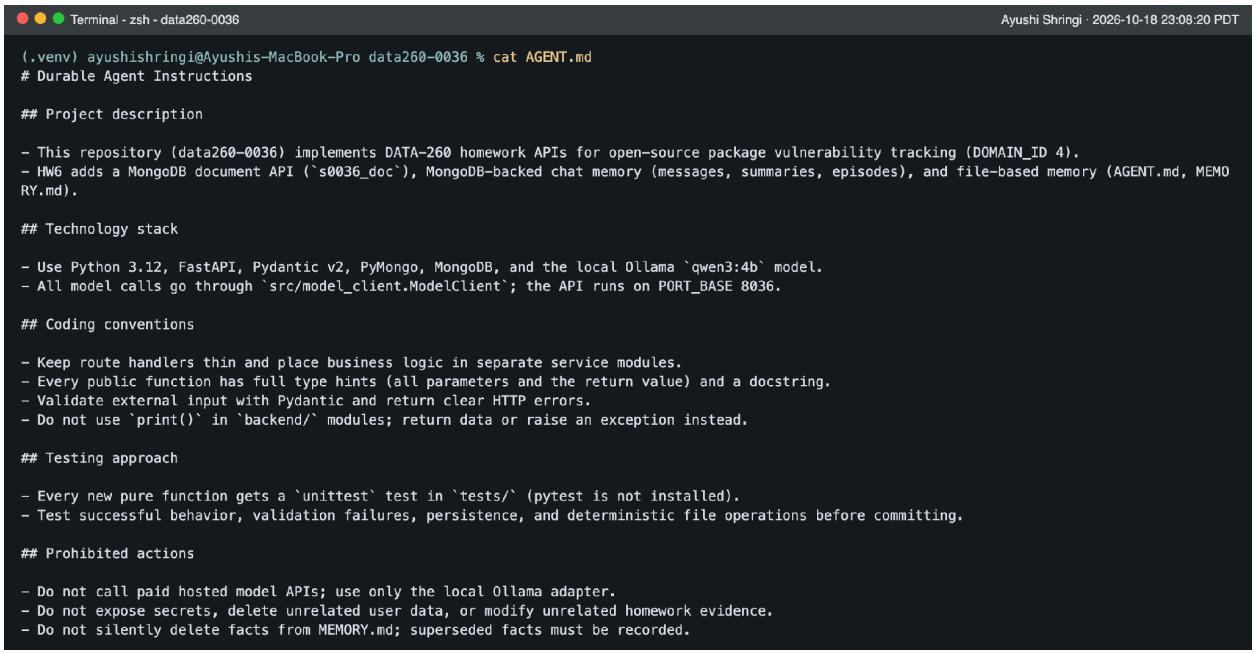
My first Part 3 results were invalid. The experiment script passed the setting only as a label, and the API was running with the default SHORT_TERM_N=12, so all four rows measured the same configuration. That data is kept in reports/hw06/raw/superseded/ and is not used above (see AI Use).

Part 4 - File-Based Memory and Consolidation

4.1 Memory files and final AGENT.md

AGENT.md (extended from HW1) stores durable project and agent instructions. It now has 12 instructions covering all five categories. MEMORY.md stores durable facts learned from conversations, one per line, as - [YYYY-MM-DD] key: value. Keys make contradictions detectable, and dates make them resolvable.

Final AGENT.md.



4.2 File loading, prompt construction and consolidation checks

```
# backend/hw6_file_memory.py

def load_memory_files() -> dict[str, str]:
    ensure_memory_files()
    return {
        "agent": AGENT_PATH.read_text(encoding="utf-8"),
        "memory": MEMORY_PATH.read_text(encoding="utf-8"),
    }

# backend/hw6_file_memory.py, line 71
def _content_lines(text: str) -> list[str]:
    return [
        line.strip()
        for line in text.splitlines()
        if line.strip() and not line.lstrip().startswith("#") and not line.lstrip().startswith("<!--")
    ]
```

```
# backend/hw6_file_memory.py
def memory_line_count() -> int:
    return len(_content_lines(load_memory_files()["memory"]))

# backend/hw6_file_memory.py

def last_consolidated_at() -> datetime | None:
    match = CONSOLIDATED_RE.search(load_memory_files()["memory"])
    if not match:
        return None
    try:
        value = datetime.fromisoformat(match.group(1).strip().replace("Z", "+00:00"))
    except ValueError:
        return None
    return value if value.tzinfo else value.replace(tzinfo=timezone.utc)

# backend/hw6_file_memory.py
def consolidation_required() -> bool:
    timestamp = last_consolidated_at()
    return memory_line_count() > 40 or timestamp is None or _utc_now() - timestamp >
timedelta(hours=24)

# backend/hw6_file_memory.py
def file_memory_context() -> str:
    files = load_memory_files()
    return f"AGENT.md instructions:\n{files['agent']}\n\nMEMORY.md durable
facts:\n{files['memory']}"
```

file_memory_context() is added to the system prompt by assemble_prompt (Part 2). Every normal chat call consolidation_required() (more than 40 non-empty, non-comment lines, or a recorded consolidation time older than 24 hours) and runs compact_memory() automatically when it returns true.

4.3 Memory commands

```
# backend/hw6_file_memory.py
def handle_file_memory_command(
    command: str,
    user_id: str,
    session_id: str | None,
    model_client: ModelClient,
) -> dict[str, Any] | None:
    normalized = command.strip().casefold()
    if normalized == "/memory":
        files = load_memory_files()
```

```
        return {
            "message": "Current file-based memory",
            "agent": files["agent"],
            "memory": files["memory"],
            "memory_line_count": memory_line_count(),
            "last_consolidated_at": str(last_consolidated_at()),
            "consolidation_required": consolidation_required(),
        }
    if normalized == "/compact":
        return compact_memory()
    if normalized == "/dream":
        return dream_memory(user_id, session_id, model_client)
    if normalized == "/help":
        return {"message": help_text()}
    return None

# backend/hw6_file_memory.py
def compact_memory() -> dict[str, Any]:
    """Consolidate MEMORY.md in place and report fact-line counts before and after."""

    files = load_memory_files()
    text, stats = consolidate_memory_text(files["memory"])
    MEMORY_PATH.write_text(text, encoding="utf-8")
    return {
        "message": (
            f"MEMORY.md consolidated: {stats['before']} -> {stats['after']} memory lines "
            f"({stats['file_lines_before']} -> {stats['file_lines_after']} file lines)"
        ),
        "message_count_before": stats["before"],
        "message_count_after": stats["after"],
        "file_lines_before": stats["file_lines_before"],
        "file_lines_after": stats["file_lines_after"],
        "duplicates_merged": stats["duplicates_merged"],
        "contradictions_resolved": stats["contradictions_resolved"],
        "resolutions": stats["resolutions"],
        "consolidation_required": consolidation_required(),
    }

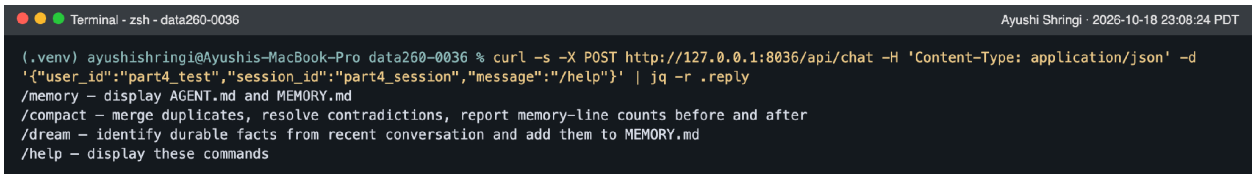
# backend/hw6_file_memory.py
def dream_memory(
    user_id: str,
    session_id: str | None,
    model_client: ModelClient,
) -> dict[str, Any]:
```

```
    files = load_memory_files()
    rows = list(
        get_messages_collection()
        .find({"user_id": user_id, "session_id": session_id, "role": "user"})
        .sort("created_at", -1)
        .limit(10)
    )
    rows.reverse()
    transcript = "\n".join(item["content"] for item in rows if not item["content"].startswith("/"))
    if not transcript:
        return {"message": "No recent user conversation is available for /dream.", "facts_added": []}
    result = model_client.complete([
        {
            "role": "system",
            "content": (
                "Identify up to three durable, useful facts from the conversation. "
                "Return only short markdown bullet lines. Do not invent facts."
            ),
        },
        {"role": "user", "content": f"Conversation:\n{transcript}\n\nExisting
memory:\n{files['memory']}"},
    ])
    existing = {_normalize(fact["value"]) for fact in parse_memory_facts(files["memory"])[0]}
    today = _utc_now().date().isoformat()
    facts = []
    for line in result.text.splitlines():
        value = line.strip().lstrip("-*").strip()
        if line.strip().startswith(("-", "*")) and value and _normalize(value) not in existing:
            facts.append(f"- [{today}] {value}")
    facts = facts[:3]
    if not facts:
        return {"message": "No new durable facts were identified.", "facts_added": []}
    with MEMORY_PATH.open("a", encoding="utf-8") as stream:
        stream.write("\n" + "\n".join(facts) + "\n")
    return {"message": "Durable facts added to MEMORY.md", "facts_added": facts}

# backend/hw6_file_memory.py
def help_text() -> str:
    return (
        "/memory — display AGENT.md and MEMORY.md\n"
        "/compact — merge duplicates, resolve contradictions, report memory-line counts before
and after\n"
        "/dream — identify durable facts from recent conversation and add them to
MEMORY.md\n"
```

```
    "/help — display these commands"
  )
```

/help explains the four commands.



/memory with the 55-line file loaded: 53 memory lines, consolidation_required = true.



/compact reports the count before and after (53 -> 34 memory lines, 55 -> 39 file lines).



/dream extracts durable facts from the recent conversation and appends them to MEMORY.md.



4.4 Verifying a generated function against AGENT.md

audit_resolutions() was generated with an AI assistant while AGENT.md was in its context. scripts/check_agent_rules.py then verifies it against eight AGENT.md rules, each quoted verbatim from the file:

- full type hints and a docstring (checked by parsing the code)

- no print() in backend modules
- no file writes
- no hosted model API
- no hard-coded port other than 8036
- a unittest exists in tests/, and the tests pass

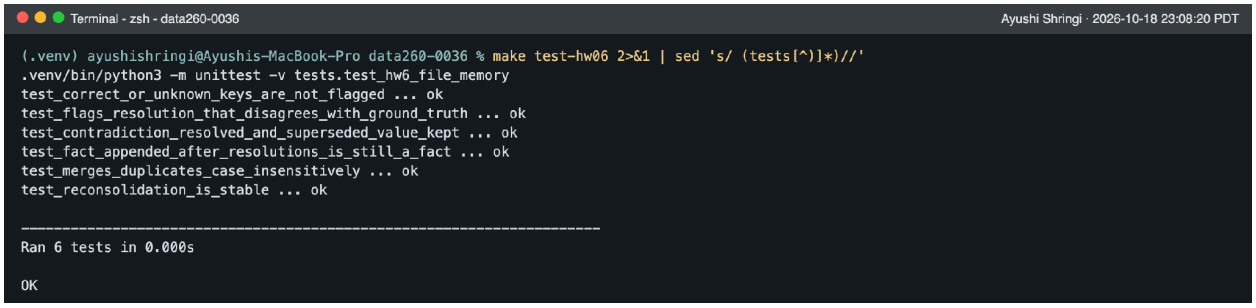
```
# backend/hw6_file_memory.py
def audit_resolutions(
    resolutions: list[dict[str, Any]],
    ground_truth: dict[str, str],
) -> list[dict[str, str]]:
    """Return the contradiction resolutions whose kept value disagrees with ground truth.

    The newest-wins rule is only a heuristic, so each resolved key that has an
    authoritative value (for example from project configuration) is checked.
    """

    incorrect: list[dict[str, str]] = []
    for item in resolutions:
        expected = ground_truth.get(item["key"])
        if expected is not None and _normalize(item["kept"]) != _normalize(expected):
            incorrect.append({"key": item["key"], "kept": item["kept"], "expected": expected})
    return incorrect
```

All eight AGENT.md rule checks pass for audit_resolutions.

Unit tests for consolidation and the audit.



4.5 Consolidation test

The 55-line MEMORY.md (reports/hw06/raw/MEMORY_original.md) contains 19 duplicated facts, both exact and case/punctuation variants. It also contains four deliberate contradictions: port, ollama_model, short_term_n_default and summarize_every_user_msgs. Consolidation works in three steps:

- Duplicates are merged into one line with a '(seen Nx)' count.
- For each contradicted key, the newest dated value is kept.
- Every superseded value is written under '## Resolved contradictions', so nothing is deleted silently.

The original and consolidated versions were committed separately (commits 2808f60 and 4e35837).

```
# backend/hw6_file_memory.py
def consolidate_memory_text(memory_text: str) -> tuple[str, dict[str, Any]]:
    """Merge duplicate facts and resolve contradictions without silent deletion.

    Duplicates (same key and value, ignoring case and spacing) are merged into
    one line with a "(seen Nx)" count. Facts that share a key but disagree are
    contradictions: the newest dated fact is kept and every superseded value is
    recorded under "## Resolved contradictions", so nothing disappears silently.
    """

    facts, previous_resolutions = parse_memory_facts(memory_text)
    before = len(_content_lines(memory_text))
    merged: dict[tuple[str | None, str], dict[str, Any]] = {}
    duplicates_merged = 0
    for fact in facts:
        identity = (fact["key"], _normalize(fact["value"]))
        if identity in merged:
            kept = merged[identity]
            kept["seen"] += fact["seen"]
            kept["date"] = max(kept["date"], fact["date"])
            duplicates_merged += 1
        else:
            merged[identity] = dict(fact)

    by_key: dict[str, list[dict[str, Any]]] = {}
    for fact in merged.values():
        if fact["key"]:
            by_key.setdefault(fact["key"], []).append(fact)

    resolutions: list[dict[str, Any]] = []
    superseded_ids: set[int] = set()
    for key, candidates in by_key.items():
        if len(candidates) < 2:
            continue
        ordered = sorted(candidates, key=lambda item: (item["date"], item["order"]))
        winner, losers = ordered[-1], ordered[:-1]
        superseded_ids.update(id(item) for item in losers)
        resolutions.append({
            "key": key,
            "kept": winner["value"],
            "kept_date": winner["date"],
            "superseded": [{"value": item["value"], "date": item["date"]} for item in losers],
```

```
            "rule": "newest dated fact wins",
        })

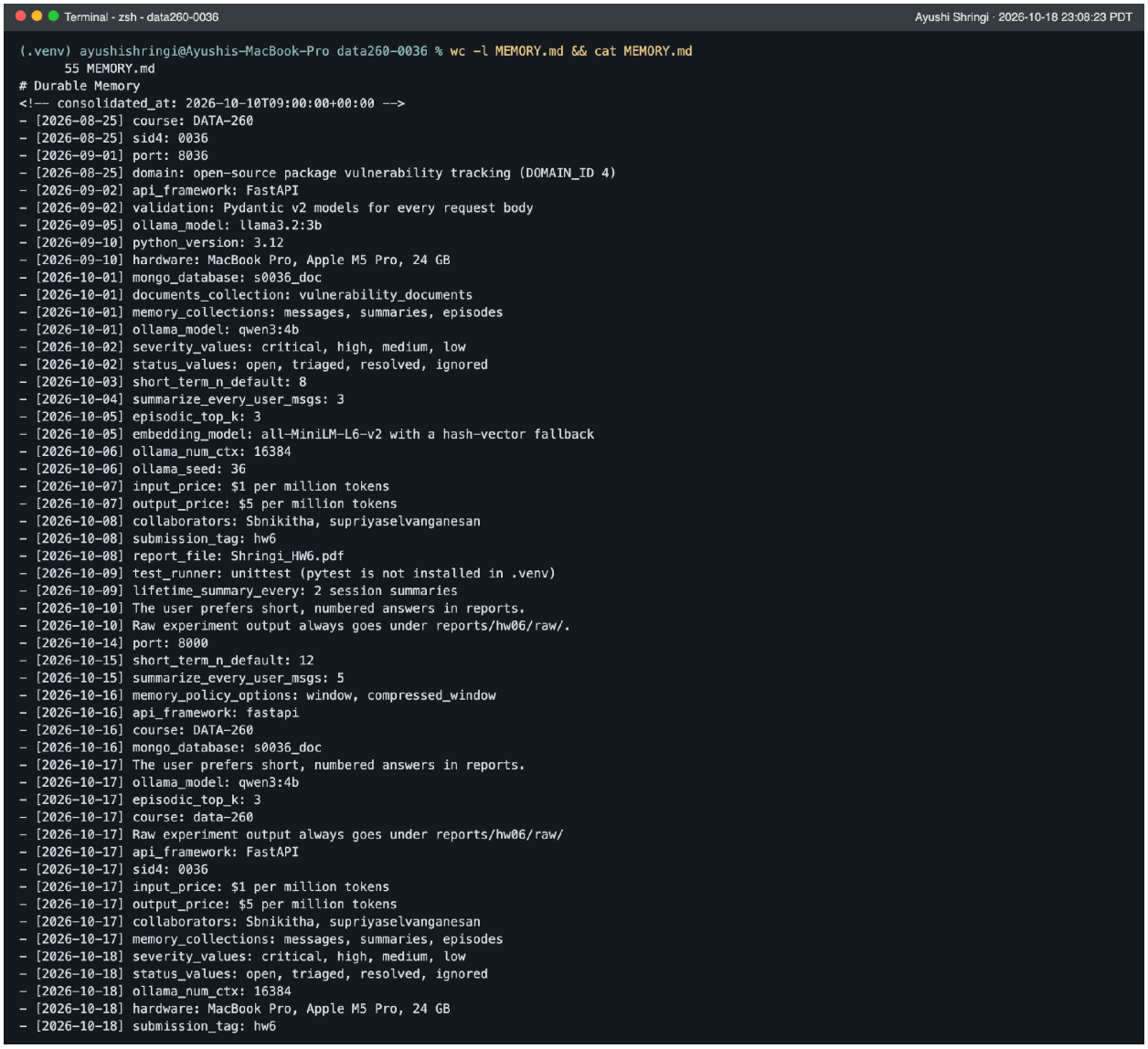
    kept_facts = sorted(
        (fact for fact in merged.values() if id(fact) not in superseded_ids),
        key=lambda item: item["order"],
    )
    resolution_lines = list(previous_resolutions)
    for item in resolutions:
        old = "; ".join(f'"{loser["value"]}" [{loser["date"] or "undated"}]' for loser in
item["superseded"])
        resolution_lines.append(
            f'- {item["key"]}: kept "{item["kept"]}" [{item["kept_date"] or "undated"}]; superseded {old}'
        )

    lines = ["# Durable Memory", f"<!-- consolidated_at: {_timestamp()} -->", "## Facts"]
    lines += [_format_fact(fact) for fact in kept_facts]
    if resolution_lines:
        lines += ["", RESOLVED_HEADING] + resolution_lines
    text = "\n".join(lines) + "\n"
    return text, {
        "before": before,
        "after": len(_content_lines(text)),
        "file_lines_before": len(memory_text.splitlines()),
        "file_lines_after": len(text.splitlines()),
        "duplicates_merged": duplicates_merged,
        "contradictions_resolved": len(resolutions),
        "resolutions": resolutions,
    }
```

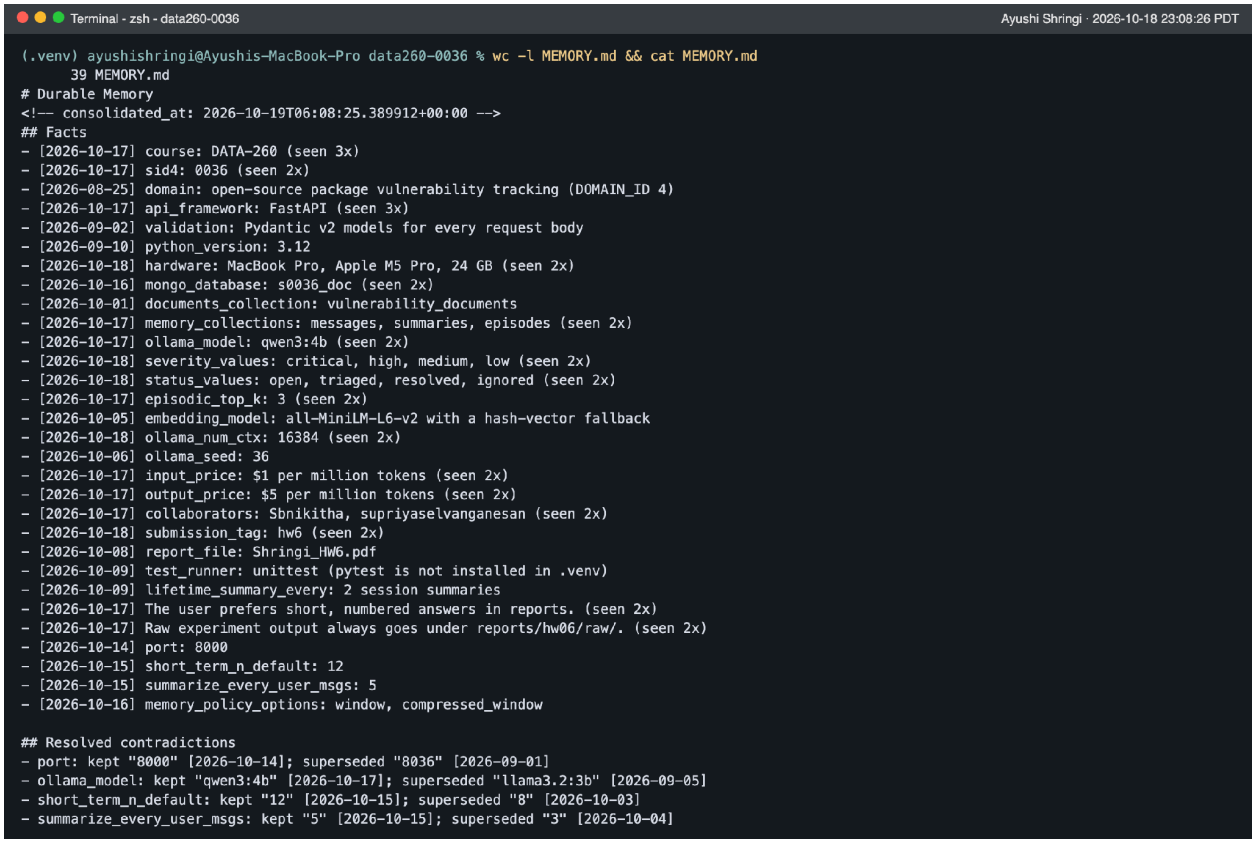
Consolidation result: 55 -> 39 lines, 19 duplicates merged, 4 contradictions resolved, 1 identified as resolved incorrectly.



Original 55-line MEMORY.md.



Consolidated MEMORY.md (39 lines) with the resolution record.

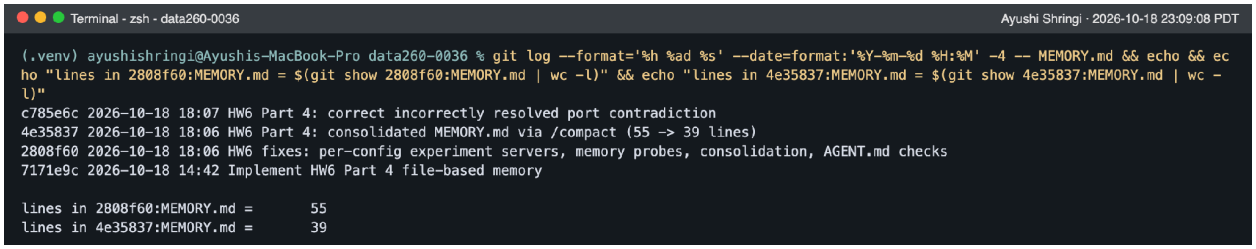


Contradiction resolved incorrectly. The newest-wins rule kept port: 8000 (dated 2026-10-14, copied from a FastAPI tutorial) over port: 8036. PORT_BASE is fixed at 8036, so this resolution is wrong. audit_resolutions() compares each resolved key with authoritative values from the project configuration, and it flags this one. I corrected it by re-asserting port: 8036 and running /compact again; the resolution log now records both decisions.

Correcting the wrong resolution.



Both versions of MEMORY.md are committed to Git.



4.6 Comparing MongoDB memory and file-based memory

Criterion	MongoDB memory (Part 2)	File-based memory (AGENT.md / MEMORY.md)
Inspectability	Needs a client (Compass, mongosh or the API). Embeddings are opaque float arrays, and summaries are model-written.	Plain Markdown that can be read and edited by hand. Every change is visible in git diff and history.
Persistence after a restart	Survives API restarts because the data lives in mongod. Edits leave no history.	Survives restarts and is versioned in Git, so any earlier state can be restored.
Scalability across many sessions	Scales well: data is indexed by user_id and session_id, and window queries and retrieval are per user.	Poor: one global file goes into every prompt (about 900 tokens here), with no per-user scoping, and it must be consolidated to stay small.
Facts that must not be forgotten	Risky: facts can fall out of the window, be paraphrased away in summaries, or be missed by top-k retrieval.	Better: always in the prompt and auditable. Consolidation can still pick the wrong side (the port example), so critical facts need review or a check against configuration.

Use MongoDB for conversation history and episodic recall, and keep a small, reviewed set of must-not-forget project rules and facts in the Markdown files.

make verify-hw06: all checks pass.



AI Use

1. What I used AI for and what I did myself. I used an AI coding assistant to help write parts of the code, to debug problems. I designed the domain mapping, the memory probes and the cost method, ran the experiments, and checked all results before using them.

2. One AI output that was wrong. The first experiment script did not actually change the memory settings. It only labeled the runs, so all four "settings" in my first results table really used the same configuration (N=12).

3. How I found it. The N=4 run used about the same number of short-term tokens per turn as the N=30 run, which should be impossible for a 4-message window.

4. What I changed. The script now starts a fresh server for each setting and stops if the server reports a different setting. I re-ran all four settings, and the token counts now grow with the window size as expected.